

DOCUMENTATION

1. Project Information

1.1 Project Title:

Development of Smart Examination Monitoring platform with Integrity Analysis & Reporting System

1.2 Project Overview

ExamGuard is a web-based online examination monitoring and integrity analytics platform developed to assist invigilators in monitoring student examination sessions.

The system combines **real-time face detection, browser activity monitoring, event logging, rule-based integrity scoring, risk classification, behavioral analytics, incident management, and AI-assisted report generation.**

The platform consists of a Flask-based examination and monitoring application and a Streamlit-based analytics dashboard. Monitoring events generated during an examination are recorded and processed to produce structured behavioral information for invigilator review.

The AI component is used specifically for generating natural-language integrity reports from the information already produced by the system. It does not independently calculate integrity scores or determine the official risk classification. This follows the project's intended design as an invigilator-assistance system rather than an automated disciplinary system.

1.3 Problem Statement

Online examinations make continuous monitoring difficult, particularly when an invigilator needs to observe multiple students simultaneously.

Events such as a student temporarily moving out of the camera's view, multiple faces appearing in the camera frame, or switching away from the examination tab can be difficult to monitor and record consistently.

ExamGuard addresses this problem by automatically recording observable examination events and converting them into structured information such as event counts, integrity scores, risk classifications, and behavioral analytics.

The resulting information is presented to the invigilator through dashboards and AI-generated reports, allowing the recorded information to be reviewed more efficiently.

1.4 Objectives

The main objectives of the ExamGuard project are:

1. To provide student registration and authentication for online examinations.
2. To create and manage examination sessions.
3. To monitor student face presence using computer vision.
4. To detect multiple faces during an examination.
5. To monitor browser activity such as tab switching.
6. To record examination monitoring events.
7. To calculate an integrity score using predefined rules.
8. To classify students into defined risk levels.
9. To analyze student behavioral patterns using data-science techniques.
10. To maintain incident records using a database.
11. To generate AI-assisted integrity reports for invigilators.
12. To provide a Flask-based invigilator interface.
13. To provide a Streamlit-based analytics dashboard.
14. To provide CSV and JSON data export functionality.

1.5 Project Scope

The scope of ExamGuard covers the monitoring and analysis of student behavior during an online examination.

- **Authentication and Exam Session Management:** It provides secure candidate login and registration paired with initial webcam photo capture, while managing the complete examination lifecycle and submission tracking.
- **Real-Time Proctoring and Vision Tracking:** It utilizes OpenCV to continuously track candidate face presence and detect potential integrity concerns, such as candidate absence or the entry of multiple faces within the camera frame.
- **Browser Activity and Incident Logging:** The system captures client-side browser events such as tab switching and stores timestamped event information in structured logs and database records.
- **Integrity Scoring and Cohort Analytics:** It computes an algorithmic integrity score with normalized risk levels for each student and applies unsupervised machine learning algorithms like K-Means clustering and PCA to reveal cohort-level behavioral trends.
- **AI-Assisted Reporting and Review Dashboards:** It leverages a LangChain-driven LLM agent to turn raw monitoring logs into factual invigilator summaries, displaying all insights, incidents, and data exports through unified review dashboards.

1.6 Motivation

The motivation behind developing ExamGuard is to demonstrate how web development, computer vision, data analytics, machine learning, and generative AI can be integrated into a unified, practical platform. Rather than treating these domains as isolated subjects, the project showcases how they collaborate within a single operational workflow: OpenCV captures visual integrity metrics, client-side event listeners track browser focus, deterministic scoring rules compute behavioral indices, unsupervised algorithms uncover cohort trends, and modern LLMs synthesize actionable summaries.

Beyond technical integration, building ExamGuard provided direct, hands-on experience across the entire software engineering lifecycle:

- **Requirement Analysis and System Design:** Formulating the operational criteria for remote academic monitoring and structuring modular interfaces between the client, backend, persistence, and analytics layers.
- **Full-Stack Implementation and Integration:** Coordinating Flask routing, session state management, template rendering, and SQLite data operations alongside continuous camera hardware polling.
- **Testing, Debugging, and Traceability:** Resolving asynchronous browser-event transmission and systematically logging edge cases during live proctoring.
- **Cohort Data Science and Machine Learning:** Applying exploratory data analysis, visual correlation heatmaps, and K-Means clustering across synthetic and session-level event distributions to derive cohort behavioral patterns.
- **Controlled AI Integration and Documentation:** Implementing a LangChain-orchestrated GenAI agent with strict prompt guardrails to guarantee factual summarization, followed by comprehensive documentation of architectural decisions and testing methodologies.

A central design principle was establishing a clear, responsible role for artificial intelligence in sensitive evaluative settings. Rather than delegating critical decisions or punitive actions to an autonomous model, the platform uses deterministic, rule-based logic for Mark deduction, integrity scoring, and initial threshold flagging. Generative AI is deployed strictly as an assistive tool—translating structured monitoring logs, anomaly frequencies, and cluster indicators into concise, readable invigilator reports. This architecture ensures fair, auditable outcomes while saving invigilators valuable time during post-exam reviews.

2.Technologies and Tools

2.1 Programming Language

Python

Python was used as the primary programming language for ExamGuard.

It was used for:

- Flask backend development
- OpenCV-based monitoring
- Event processing
- Integrity scoring
- Data processing
- K-Means and PCA analysis
- AI integration
- Streamlit dashboard development
- Data export operations

Python was selected because it provides libraries covering web development, computer vision, data science, machine learning, and AI integration within a single ecosystem.

2.2 Web Framework

Flask

Flask was used to develop the main web application.

It handles:

- Student registration and login , Session management ,Examination workflow
- Monitoring endpoints ,Browser event handling ,Invigilator dashboard
- AI report pages and Data export routes

The Flask application acts as the main backend connecting the examination interface with the monitoring, scoring, analytics, and reporting components.

2.3 Frontend Technologies

HTML

HTML was used to create the examination and dashboard interfaces.

It provides the structure for:

- Login and registration pages
- Examination questions
- Monitoring interface
- Invigilator dashboard
- AI report display

CSS

CSS was used to style the web interfaces and improve the usability and visual presentation of the application.

JavaScript

JavaScript was used for browser-side functionality, particularly for handling browser activity and communicating examination events with the Flask backend.

2.4 Computer Vision

OpenCV

OpenCV was used for real-time webcam monitoring.

The main computer-vision functionality includes:

- Capturing the camera feed
- Detecting faces
- Determining whether a face is present
- Detecting multiple faces

Haar Cascade

A Haar Cascade classifier was used for face detection.

The system uses the detected face information to generate monitoring events such as:

Face Absent

Multiple Faces

Important: ExamGuard uses **face detection and not face recognition**. Therefore, the system does not identify or verify the identity of the person detected by the camera.

2.5 Database Architecture: SQLite

ExamGuard uses **SQLite** (examguard.db) as its lightweight relational database for storing user authentication and examination incident records.

Table Schema and Field Specifications

1. User Authentication (users table):

- **username (TEXT, Primary Key):** Unique identifier for the registered candidate.
- **password (TEXT):** Credential used for user authentication.

2. Incident Records (incidents table):

- **id (INTEGER, Primary Key, Autoincrement):** Unique identifier for each incident.
- **username (TEXT):** Identifies the candidate associated with the incident.
- **event_type (TEXT):** Type of recorded event, such as "**Tab Switched**".
- **details (TEXT):** Additional information about the recorded event.
- **timestamp (TEXT):** Date and time when the incident was recorded.
- **severity (TEXT):** Severity assigned to the recorded incident.

2.6 Data Processing: Pandas

Pandas is used for data processing and analysis in ExamGuard. It connects raw event logs with the machine-learning, visualization, AI reporting, and export components.

Primary Data Processing Operations

- **Data Ingestion:** Reads CSV files such as student_behavior.csv, student_clusters.csv, exam_sessions.csv, and event_logs.csv.
- **Session Filtering:** Filters student records and event logs according to the active examination session.
- **Event Aggregation:** Calculates metrics such as Face Absent, Multiple Faces, Tab Switches, and face presence.
- **Feature Preparation:** Prepares behavioral features for K-Means clustering and PCA analysis.
- **Data Export:** Connects SQLite incident records with Pandas for CSV and JSON report generation.

2.7 Machine Learning and Data Analytics

Scikit-learn

Scikit-learn was used for the project's behavioral analytics.

The main techniques used are:

K-Means Clustering

Used to group students according to behavioral characteristics.

Principal Component Analysis (PCA)

Used to reduce the behavioral feature space to two dimensions for visualization.

The analytics component works with features such as:

- Face Absent
- Multiple Faces
- Tab Switches
- Total Suspicious Events
- Integrity Score

2.8 Data Visualization: Matplotlib & Seaborn

Matplotlib and **Seaborn** are used to visualize ExamGuard's monitoring and behavioral data for post-exam analysis.

- **Matplotlib:** Generates K-Means/PCA visualizations and saves charts such as `kmeans_clusters.png`.
- **Seaborn:** Creates statistical and categorical visualizations, including integrity-score distributions, suspicious-event frequencies, and behavioral heatmaps.
- These visualizations help invigilators identify **behavioral patterns and anomalies** across students.

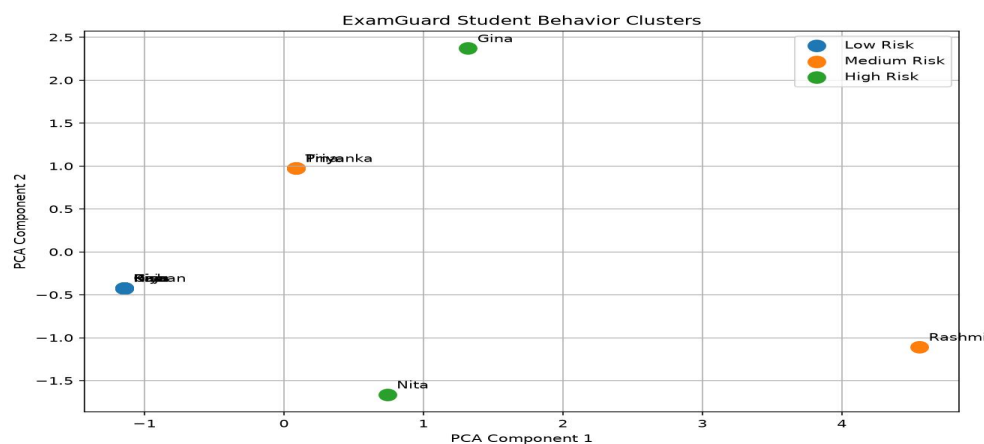


Figure 1: Student Behavioral Clusters using K-Means and PCA

The PCA projection provides a two-dimensional visualization of student behavioral clusters and their configured risk levels.

2.9 Artificial Intelligence: LangChain & Google Gemini API

ExamGuard integrates Generative AI to convert structured behavioral data and recorded examination events into clear, natural-language reports for post-examination review.

LangChain Framework

- **Pipeline Orchestration:** LangChain connects processed student metrics, event information, and structured prompts with the Google Gemini language model.
- **Constrained Prompt Engineering:** ChatPromptTemplate defines rules that restrict the model to recorded information. The prompt prevents unsupported claims, requires the use of **face detection** rather than face recognition, prevents inference of student intent, and preserves the official integrity score and risk level.
- **Structured Report Generation:** The AI output follows a consistent format containing **Summary, Recorded Evidence, Assessment, and Recommendation** sections.

Google Gemini API Integration

- **Model:** ExamGuard uses the gemini-3.6-flash model through the ChatGoogleGenerativeAI interface.
- **Assistive Role:** Gemini is used only for natural-language report generation. It receives structured metrics such as face absence, multiple-face detections, tab switches, and recorded events. It does **not** calculate or modify the integrity score or official risk classification.

Dependency Specifications

- google-genai: **2.24.0**
- langchain-google-genai: **4.4.0**



Figure 2: AI-Generated Integrity Report

The AI-generated report summarizes the recorded examination events, integrity score, and risk classification for invigilator review.

2.10 Analytics Dashboard: Streamlit

Streamlit was used to build an interactive analytics dashboard for invigilators. It consumes data generated by the Flask application and analytics modules for post-exam review.

Key Features:

- **Risk & Integrity Analysis:** Displays Low, Medium, and High Risk counts and integrity scores.
- **Event Analysis:** Shows recorded Face Absent, Multiple Faces, and Tab Switch events.
- **Student Profiling:** Allows selection and review of individual student data.
- **K-Means/PCA Visualization:** Displays behavioral clustering results.
- **Incident Review:** Displays recorded incident details and metadata.
- **AI Report:** Displays the generated AI integrity report.
- **Data Export:** Provides downloads for event logs, integrity scores, cluster results, incident records, and complete session JSON.

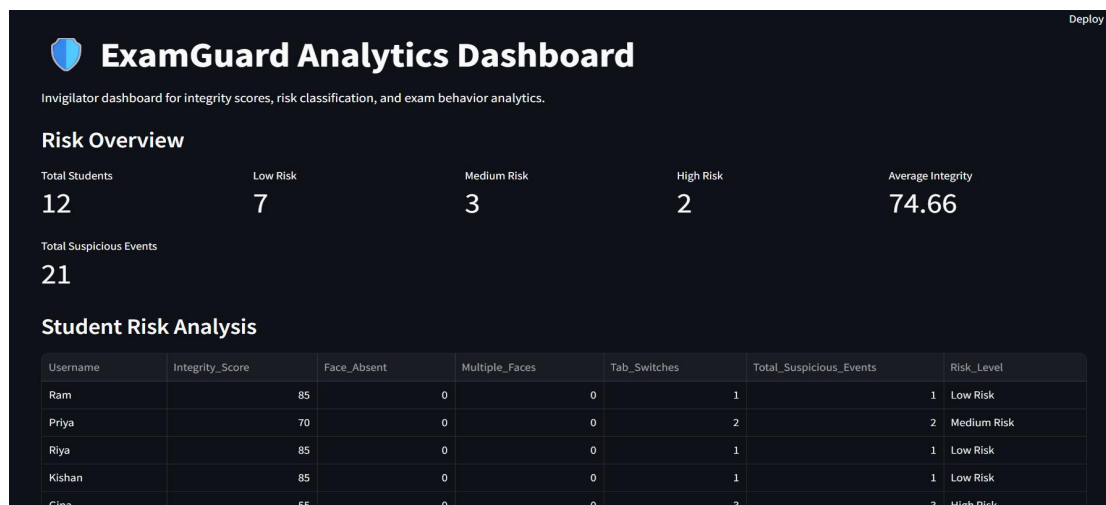


Figure 3: ExamGuard Streamlit Analytics Dashboard

The dashboard provides invigilators with student risk analysis, behavioral analytics, incident information, AI reports, and data export controls.

2.11 Data Formats

CSV

CSV files are used for structured tabular information such as:

- Event logs
- Student behavior
- Cluster results
- Integrity scores
- Incident records

JSON

JSON is used for exporting complete session information in a structured format.

TXT

Text format is used for storing and downloading generated AI integrity reports.

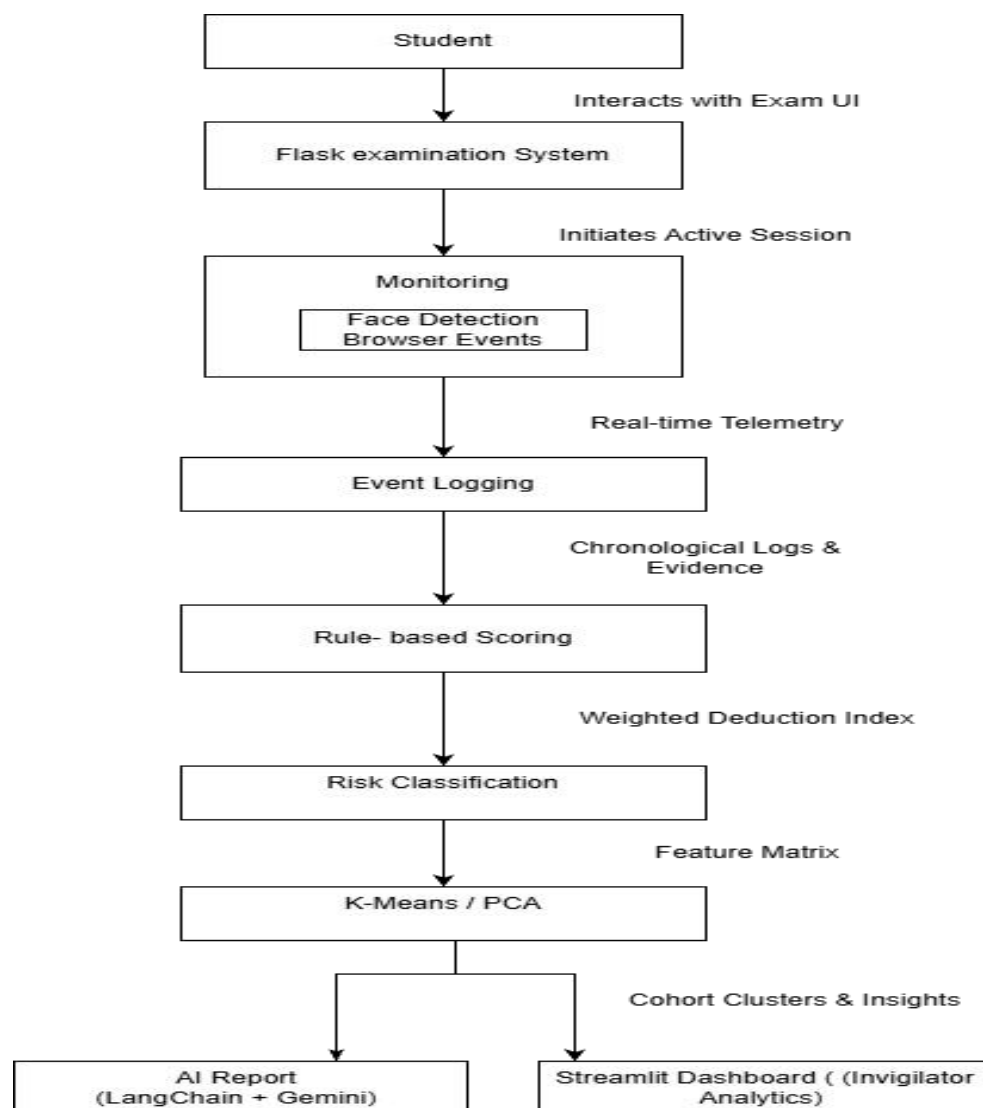
2.12 Development Tools

The project was developed and tested using a local Python development environment.

The development process involved:

- VS Code
- Python virtual/development environment
- PowerShell/terminal
- Flask development server , Browser-based testing
- Streamlit development server
- Google AI Studio for Gemini API configuration

3 . System Architecture & Workflow



Step-by-Step Architectural Flow

1. Student & Flask Examination System:

The candidate registers, logs into the platform, and accesses the examination interface managed by the Flask application and session controller.

2. Monitoring (Face Detection & Browser Events):

Monitoring runs during the examination. OpenCV detects face presence and multiple faces through the webcam, while browser-side JavaScript detects events such as tab switching.

3. Event Logging:

Detected monitoring events, timestamps, and event details are recorded in structured CSV logs and SQLite incident records.

4. Rule-Based Scoring:

The deterministic scoring engine applies predefined penalty weights to recorded events starting from a baseline score of 100 and calculates the candidate's integrity score.

5. Risk Classification:

Students are assigned **Low Risk, Medium Risk, or High Risk** based on the predefined integrity-score thresholds.

6. K-Means / PCA Behavioral Analysis:

Behavioral metrics are processed using K-Means clustering to identify groups of students with similar behavioral patterns. PCA is used to reduce the feature space for two-dimensional visualization.

7. Output Layer (AI Report & Dashboards):

- **AI Report:** The LangChain pipeline passes structured student metrics and recorded events to Google Gemini (gemini-3.6-flash) to generate a natural-language integrity report. The AI does not calculate or modify the official score or risk level.
- **Flask Invigilator Dashboard:** Displays student risk information, integrity scores, behavioral analytics, and AI report generation options.
- **Streamlit Dashboard:** Provides interactive analytics including risk profiles, integrity scores, event analysis, K-Means/PCA visualization, incident records, AI reports, and data export options.

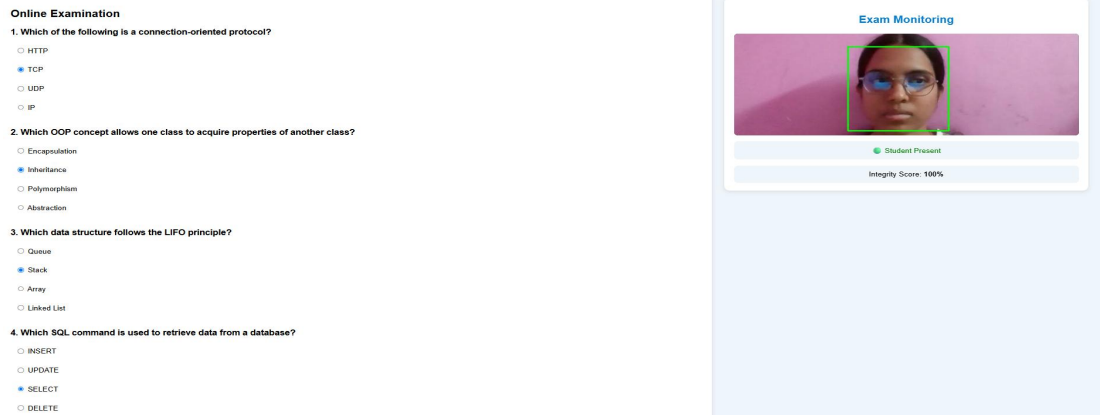


Figure 4: Online Examination Interface with Real-Time Face Monitoring

The examination interface allows the student to attend the test while the webcam-based monitoring component observes face presence during the session.

Student	Exam Score	Face Absent	Multiple Faces	Tab Switches	Suspicious Events	Integrity Score	Risk Level	AI Report
Ram	Not Submitted	0	0	1	1	85.0	Low Risk	<button>Generate AI Report</button>
Riya	4/4	0	0	1	1	85.0	Low Risk	<button>Generate AI Report</button>
Kishan	0/4	0	0	1	1	85.0	Low Risk	<button>Generate AI Report</button>
Gina	4/4	0	0	3	3	55.0	High Risk	<button>Generate AI Report</button>
Raju	Not Submitted	0	0	1	1	95.5	Low Risk	<button>Generate AI Report</button>
Hira	4/4	0	0	1	1	89.43	Low Risk	<button>Generate AI Report</button>
Tina	4/4	0	0	2	2	82.97	Low Risk	<button>Generate AI Report</button>
Nita	4/4	1	0	1	2	34.17	High Risk	<button>Generate AI Report</button>
Priyanka	3/4	0	0	2	2	63.27	Medium Risk	<button>Generate AI Report</button>
Gia	3/4	0	0	1	1	86.58	Low Risk	<button>Generate AI Report</button>
Rashmi	4/4	1	1	2	4	63.97	Medium Risk	<button>Generate AI Report</button>
Priya	4/4	0	0	2	2	90.59	Low Risk	<button>Generate AI Report</button>

Figure 5: ExamGuard Flask Invigilator Dashboard – Student Risk Analysis

The dashboard presents student-wise examination results, integrity scores, detected events, and risk levels, with an option to generate an AI-assisted integrity report for invigilator review.

4. Development Challenges and Solutions

During the development of ExamGuard, several technical challenges were encountered while integrating the different modules. These challenges helped in understanding debugging, integration, API limitations, and system stability.

Challenge / Issue	Solution / Action Taken	Learning
Gemini	API The Gemini integration was tested	Learned about model

Challenge / Issue	Solution / Action Taken	Learning
model/configuration issues	and updated to use a compatible model configuration.	compatibility and API-based development.
Gemini API quota exhaustion	Further unnecessary API calls were avoided after the report-generation functionality had been successfully validated.	Learned about API quotas and responsible API usage during development.
K-Means/PCA visualization error	An outdated cluster-to-risk mapping caused an error. The visualization was updated to use the current Risk_Level data.	Learned to debug data-processing and visualization pipelines.
Screenshot evidence caused monitoring instability	Screenshot capture was intentionally deferred because it caused the camera/monitoring experience to freeze.	Learned that system stability should be prioritized over adding a feature.
JSON export contained missing values	Missing Pandas values were converted to JSON-compatible null values before export.	Learned about data serialization and handling missing values.
Multiple modules had to remain consistent	Flask, CSV analytics, AI reports, and Streamlit were tested against the same student data.	Learned the importance of end-to-end data consistency.

Key Development Lesson

The main learning from these challenges was that building the project was not only about implementing individual features, but also about **integrating and testing the complete system while maintaining stability and consistency**.

5. Testing and Results

The completed ExamGuard system was tested module-by-module and through an end-to-end workflow to verify that monitoring, scoring, analytics, AI reporting, dashboards, and exports worked consistently.

Test Case	Expected Result	Result
Student registration/login	Student can authenticate and access the examination	✓ Passed
Exam session	Examination session is created and maintained correctly	✓ Passed
Face monitoring	Face presence is detected during the examination	✓ Passed
Face absent	Face Absent event is recorded and scoring rule is applied	✓ Passed
Multiple faces	Multiple-face event is detected and recorded	✓

Test Case	Expected Result	Result
		Passed
Tab switching	Tab Switched event is recorded and scoring rule is applied	✓ Passed
Combined events	Multiple events produce the corresponding cumulative score	✓ Passed
Risk classification	Score is mapped to the appropriate configured risk level	✓ Passed
K-Means/PCA	Behavioral analysis and visualization are generated	✓ Passed
Flask dashboard	Student analytics and risk information are displayed	✓ Passed
Streamlit dashboard	Analytics and student-level information are displayed	✓ Passed
Data consistency	Flask, Streamlit and CSV results show matching values	✓ Passed
Incident logging	Incident records are stored and retrieved correctly	✓ Passed
AI report	Report contains correct score, risk level and recorded evidence	✓ Passed
CSV exports	Exported files download and contain expected data	✓ Passed
JSON export	Complete session data is exported as valid JSON	✓ Passed
Edge cases	Clean sessions, zero scores, missing reports/data are handled without crashing	✓ Passed

Testing Outcome

The testing confirmed that the major implemented components of ExamGuard work together as expected. Particular attention was given to **data consistency**, ensuring that the integrity score and risk level shown in the Flask dashboard, Streamlit dashboard, and generated analytics data remain consistent.

The AI report was also validated to ensure that it uses the recorded monitoring information without independently calculating scores, changing the official risk classification, or making unsupported claims.

6. Learning Outcomes, Limitations and Future Scope

Learning Outcomes

- Gained practical experience in **Flask, OpenCV, SQLite, Pandas, Scikit-learn and Streamlit**.

- Learned to integrate **LangChain and Gemini API** for AI-assisted report generation.
- Learned **K-Means clustering and PCA** for behavioral analytics.
- Improved debugging, API integration, data processing and end-to-end testing skills.
- Learned the importance of separating **rule-based decisions from AI-generated explanations**.

Limitations

- Face monitoring uses **face detection**, not face recognition.
- Monitoring events do not independently establish academic misconduct.
- Screenshot evidence capture was **deferred due to monitoring stability issues**.
- Gemini report generation depends on API availability and quota.
- Risk classification currently uses predefined rule-based thresholds.

Future Scope

- More robust face-monitoring techniques.
- Stable screenshot/evidence capture.
- Improved session-level evidence management.
- Enhanced behavioral analytics.
- Stronger authentication and role-based access.
- Cloud deployment and production database integration.

Conclusion

ExamGuard successfully integrates online examination monitoring, rule-based integrity scoring, behavioral analytics, incident management, AI-assisted reporting and dashboard-based analysis into a single platform for invigilator assistance.